CourierLedger M1 Database Design Specification

Technical Correction Amendment v0.4

A targeted correction amendment to the Database Design Specification v0.1 (36 pages), as amended by Technical Correction Amendment v0.2 (CL-01 to CL-12) and Technical Correction Amendment v0.3 (CL-13 to CL-17). This amendment supersedes only the sections, tables, constraints and test scenarios it explicitly names. Everything else in v0.1, v0.2 and v0.3 stands unchanged.

> **Scope guard.** This document contains no application code and no executable migrations. SQL fragments, index definitions and update predicates are illustrative design notation, marked ADAPT, exactly as in v0.1 Section E, v0.2 and v0.3. No courier rate, fee, statement format or contract semantic is invented. V2.2.1 formulas and identities, integer BIGINT poisha, per-component half-up rounding, NULL versus verified zero, and the explicit D_C confirmation-plus-evidence rule are preserved without modification. CL-01 to CL-17 are preserved unless a concrete contradiction is demonstrated - each finding below names the contradiction it corrects.

What this amendment does

- Resolves five follow-up findings, CL-18 to CL-22, each in the established six-part form: original problem, exact change, affected section and table, constraint or transaction-level enforcement, updated test scenario, and a resolved / partially resolved / still-open verdict.
- Adds no table. The design total stays at 31 tables. One column is added - a batch reference on the CL-16 inputs table - together with the unique keys that let composite foreign keys carry it (CL-21).
- Ends with a consolidated test-change table, an invariant traceability delta, a change log from v0.3 to v0.4, the remaining blockers, and a statement on schema freeze.

Reading rule. Where this amendment and v0.1, v0.2 or v0.3 disagree, this amendment wins for the named item only. Where it is silent, v0.1 as amended by v0.2 and v0.3 applies.

> **Gating - unchanged.** M1 production implementation remains gated behind the Day 7 and Day 14 market-validation gates, merchant-confirmed courier evidence, a Docker + PostgreSQL verification environment, and explicit production-readiness approval. This amendment authorises none of those steps, and no test in it is claimed to have passed.

Baseline	Database Design Specification v0.1 (36 pages)
Prior amendments	Technical Correction Amendment v0.2 (CL-01 to CL-12); v0.3 (CL-13 to CL-17)
This amendment	v0.4 - CL-18 to CL-22
Financial authority	Specification V2.2.1 and the P0 final clarification, unchanged

Amendment at a glance

Five findings. Each one corrects a place where two earlier statements, both reasonable on their own, cannot both be executed as written: a trigger that would reject the very test data meant to exercise detection (CL-18), two clocks on one table with no rule for which one governs deletion (CL-19), a lock that covers existing rows but not the insert racing it (CL-20), a tenant check that proves the right workspace but not the right settlement batch (CL-21), and a validation mechanism whose ability to see the rows it validates was asserted but never specified (CL-22).

ID	Subject	Affected locations	Verdict
CL-18	CL-13 trigger enforcement versus malformed-history testing: trigger scope and the legacy-data fixture	v0.3 CL-13 sections 2(c), 4; test T-26(e)	Design-resolved; verification outstanding
CL-19	CL-14 idempotency purge and lease lifecycle: purge eligibility and exact conditional-update predicates	v0.3 CL-14 sections 2, 4; v0.2 CL-04; test T-27	Design-resolved; verification outstanding
CL-20	CL-15 shared-user anonymisation serialisation: per-user lock protocol for all membership mutations and Stage 2	v0.3 CL-15 sections 2, 4; tests T-25, T-30; OD-16, OD-17	Mechanics design-resolved; OD-16 and OD-17 still open
CL-21	CL-16 same-batch input integrity and current variance validity: composite keys and input-set equality	v0.3 CL-16 sections 2, 4; v0.1 Section C; tests T-28, new T-31	Design-resolved; verification outstanding
CL-22	CL-17 trigger execution context under FORCE RLS: owner, derivation of merchant_id, scoped visibility, fail-closed rule	v0.3 CL-17 section 2; v0.1 Section E; new test T-32	Design-resolved; verification outstanding

What changes structurally - and what does not

- **No new tables.** The total stays at 31. CL-21 was required to strengthen the two CL-16 tables without adding a third, and it does: the mechanism is keys and a derivation rule, not storage.
- **One column added:** `settlement_batch_id` on `settlement_variance_calculation_inputs` (CL-21), plus one supporting unique key each on `settlement_variance_calculations`, `bank_receipts` and `settlement_allocations` so composite foreign keys can carry the batch.
- **No new statuses, no new columns elsewhere.** CL-19 works entirely within the CL-04 / CL-14 columns (`status`, `claim_token`, `lease_expires_at`, `expires_at`, `attempt_count`). CL-20 adds a lock protocol, not a column. CL-22 adds an execution-context pattern, not a grant.
- **No formula changes.** Nothing here touches C, Δ, V_R, S, V_{cond}, V_{total}, the identities $V_{total} = C - \Delta - V_R + S$ and $V_{cond} = S - V_R$, the CL-01 five-step status derivation, or the CL-10 allocation and reversal semantics.
- **No decision is silently closed.** OD-12, OD-16 and OD-17 stay open. CL-20 and CL-21 explicitly preserve them.

> **Design resolution is not verification.** A verdict of design-resolved in this amendment means the contradiction is removed and a concrete, PostgreSQL-accurate mechanism is specified. It does not mean the mechanism has been run. No M1 implementation exists, the Docker + PostgreSQL verification environment (blocker B-3) is not established, and every mechanism named here remains unproven against a running database until that environment exists. The final section returns to what this means for schema freeze.

CL-18 - CL-13 trigger enforcement versus malformed-history testing

1. Original problem

v0.3 CL-13 contains two statements that cannot both be executed. Section 4 specifies a deferred constraint trigger on `shipment_match_decisions` that, at commit, re-derives the candidate's state and *rejects the transaction if it would leave a newly written history malformed*. Test T-26(e) then instructs the tester to *insert two roots for one candidate as the migrator role* - one ending confirmed, one rejected - and expects the inserts to succeed so that the derivation can be shown to report the candidate as conflicted.

A deferred constraint trigger does not know which role inserted the rows or why. If the trigger is active, the T-26(e) transaction is exactly the transaction section 4 says it rejects: it aborts at commit, no malformed history exists, and the derivation's conflicted case - the case that protects a reconciliation run from relying on a forked history - can never be demonstrated. If the tester disables or drops the trigger to make T-26(e) pass, the test has quietly proved the opposite of section 4, and nothing records that the production integrity mechanism was switched off to obtain the result.

Section 2(c) of CL-13 already points at the distinction without drawing it: malformed histories were described as arriving *through a pre-v0.3 data state, or a migrator-role maintenance write* - that is, as data that *already exists* when the derivation runs - while the trigger was described as policing *the application's own path*. v0.3 never stated the trigger's scope precisely enough to separate those two populations, and T-26(e) was written as if the migrator role stood outside the trigger's scope. It does not. Constraint triggers fire for every role that writes the table, subject only to the trigger being enabled - which is precisely why the fixture below, and not a role, must carry the exception.

2. Exact proposed change - two cases, kept distinct

Clarifies v0.3 CL-13 sections 2(c) and 4, and revises T-26(e). The linear-chain invariant, the candidate advisory-lock serialisation, the partial unique successor index, the composite self-FK, the deterministic derivation and the append-only model are all retained unchanged. Two cases are defined once and used consistently from here on:

Case	Definition and governing mechanism
Case A - malformation a transaction would create	The transaction writes decision rows that introduce a malformation that did not exist before it: a second root for a candidate that already has a root, a successor that does not extend the chain, or a first history that is already forked at commit. The deferred constraint trigger rejects the transaction at commit. This applies to every writing role, including the migrator role, whenever the trigger is enabled. There is no role-based exemption.
Case B - malformation that already exists	Decision rows forming a malformed history are already committed when a later transaction or read arrives - legacy data predating the trigger, a restored backup, or a deliberately corrupted test fixture. No trigger *creates* this state and no trigger is asked to repair it: triggers reject transactions; they do not rewrite history. The CL-13 derivation detects it, deterministically and read-only: multiple disagreeing heads, or multiple heads of which at least one is confirmed, derive as *conflicted* with no current decision; unanimous rejection derives as rejected. Detection never modifies a row.

The boundary between the cases is the transaction, not the role and not the intent. A malformation is Case A while the transaction that introduces it is still open, and Case B from the moment it is committed - which, with the trigger enabled, can only happen for data that entered the table outside the trigger's lifetime. That is not a loophole in the design; it is the accurate description of what a commit-time trigger can and cannot govern, and the test revision in section 5 is written to match it.

2. Exact proposed change (continued) - the trigger's scope, stated precisely

The trigger judges the delta, not the whole history. The v0.3 wording - reject if the commit *would leave a newly written history malformed* - is superseded by the following scope rule. At commit, for each candidate touched by the transaction, the trigger function separates the candidate's rows into rows written by this transaction (the delta) and rows that were already committed before it (the prior state), using design notation ADAPT: rows inserted by the current transaction are identifiable by their transaction identifier, and this identification is one of the mechanisms B-3 verification must prove. The trigger then rejects the transaction if and only if the delta *introduces* a malformation:

- the delta contains a root while a root already exists in the prior state, or the delta itself contains more than one root for a candidate with no prior state;
- a delta successor names a predecessor that is not the head implied by the prior state plus earlier delta rows, where the partial unique index has not already rejected the insert at statement time;
- the delta leaves a previously well-formed history forked.

Two consequences follow, both intended. First, a transaction that merely *extends* a pre-existing malformed history through the sanctioned resolution paths of CL-13 section 2(d) - inserting a rejecting successor naming an existing head - is not rejected for the pre-existing malformation: it introduces none, and blocking the repair path because the damage exists would make conflicted candidates unresolvable. Second, an unrelated transaction touching a different candidate is never rejected because a legacy malformation exists elsewhere in the table. Pre-existing malformation is a property the derivation reports, candidate by candidate; it is not a poison flag on the table.

The legacy-data fixture. T-26(e) is revised to build its malformed history as an explicitly controlled fixture, in a disposable test database only, by one of two permitted routes:

- **Load-before-trigger (preferred).** The fixture is loaded at the point in the test database's build where the CL-13 deferred constraint trigger has not yet been created - the same position legacy production data would occupy relative to the migration that introduces the trigger. The trigger is then created, and every subsequent assertion in T-26(e) runs with the trigger fully active.
- **Replica-mode load (test harness only).** The fixture loader, running as the migrator role in the test database only, loads the malformed rows with trigger firing suppressed for that loader session (design notation, ADAPT: a session-level trigger-suppression setting available only to the table owner), then restores normal firing before any assertion runs. The suppression is a property of the fixture loader session, is never available to the runtime role, and must never be used in a production database or in the same session that later asserts detection.

Under either route, the fixture is labelled as legacy data in the test itself, the production trigger definition is identical to the one the test runs against, and the test asserts detection by the derivation - a read - with the trigger active. What the revised test may no longer do is imply that an ordinary migrator-role insert, with the trigger enabled, commits a malformed history. T-26 gains a companion assertion (section 5, part (h)) proving exactly that it cannot.

> **What this amendment does not do.** It does not weaken the trigger to make the test possible: the trigger still rejects every transaction, from every role, that would create a malformation while it is enabled. It does not give the migrator role a standing bypass: the only route to a committed malformation is the trigger not yet existing, or a supervised fixture load in a disposable database - the same two routes by which real legacy data exists. And it does not let detection mutate anything: the derivation and the fixture assertions are read-only, and the malformed rows remain byte-identical throughout, as the append-only model already required.

3. Affected sections and tables

v0.3 CL-13 section 2(c) (the arrival routes for conflicting initial decisions - clarified to name them Case B and to remove any implication that a migrator write stands outside the trigger); v0.3 CL-13 section 4, final enforcement paragraph (the trigger's scope - superseded by the delta rule in section 2 above); Section I test T-26(e) (revised) and T-26(h) (added). Table `shipment_match_decisions`: no column, index or constraint changes. The partial unique successor index, the composite self-FK, the same-row self-reference CHECK and the append-only grant model are unchanged.

4. Constraint or transaction-level enforcement

- **Statement-time layer, unchanged.** The partial unique index still rejects a second successor at insert; the composite self-FK still rejects cross-candidate and cross-tenant supersession at insert. Neither mechanism is role-sensitive, and neither is affected by this clarification.
- **Commit-time layer, scope corrected.** The deferred constraint trigger (ADAPT) evaluates the delta rule of section 2 for each touched candidate and rejects only malformation the committing transaction introduces. Its execution context - owner, visibility under FORCE RLS, derivation of `merchant_id` from the firing rows, and the fail-closed rule - is the pattern specified once in CL-22 of this amendment, which applies to this trigger without exception.
- **Read layer, unchanged in substance.** The CL-13 derivation remains a pure function of stored rows: one root, one head, one walk for well-formed; unanimous rejection for multiply-rejected heads; conflicted otherwise. It is the sole mechanism that pronounces on Case B data, and it writes nothing.
- **Fixture discipline as an enforcement boundary.** Trigger suppression for fixture loading exists only in the test harness, only in the loader session, and only in a disposable database. No production role, migration or maintenance procedure created by this design may suppress the trigger; the CL-11 deployment check gains a corresponding verification item (see Remaining blockers).

5. Updated test scenario

T-26(e) (revised) - Pre-existing malformed history, detected read-only. Build the two-root history (one chain ending confirmed, one ending rejected, same candidate) as a legacy fixture by the load-before-trigger route, or by the replica-mode loader in the test database. Create the CL-13 trigger (route one) or restore normal firing (route two) before asserting. Expected, with the trigger fully active throughout the assertions: the derivation reports the candidate conflicted with no current decision; a run that would rely on it classifies the shipment `needs_review` with a data-quality finding; neither fixture row has been altered; and the fixture rows are identified in the test as legacy data, not as the product of an application-path insert.

T-26(h) (new) - The active trigger rejects a newly created malformation, for every role. With the trigger enabled, attempt as the runtime role, and separately as the migrator role in an ordinary (non-fixture) session, to commit a second root for a candidate that already has one: both transactions fail at commit. Attempt the same for a candidate with a pre-existing legacy malformation, where the new transaction adds a further root: the transaction fails at commit, and the legacy rows remain as loaded. Parts (a) to (d), (f) and (g) of T-26 are unchanged; part (f)'s resolution inserts remain valid under the delta rule and must still commit.

6. Status

Design-resolved; implementation verification outstanding. The contradiction is removed by scoping the trigger to the malformation a transaction introduces and by moving T-26(e)'s data to an explicitly controlled legacy fixture, with the production trigger definition untouched. The delta-identification mechanism named in section 2 is design notation (ADAPT): it must be proved in the B-3 verification environment before any part of this finding is treated as verified.

CL-19 - CL-14 idempotency purge and lease lifecycle

1. Original problem

v0.3 CL-14 separated the two clocks on `idempotency_keys` - `lease_expires_at`, the ownership horizon measured in minutes, and `expires_at`, the 24-hour replay horizon set by v0.2 CL-04 - and then stated the purge rule in a single sentence inherited from CL-04: after `expires_at`, the maintenance role may purge the key row and the same key string may be claimed fresh. That sentence is correct about completed keys and silent about everything else:

- **It does not exclude a live claim.** Read literally, a purge job keyed on `expires_at` alone could delete an `in_progress` row whose worker is still legitimately executing - the replay horizon and a long-running operation can overlap - destroying the fencing record and admitting a duplicate execution of a financial operation.
- **It does not say what happens to abandoned claims.** An `in_progress` row with an expired lease is the recovery case CL-14 exists for. Whether the purge job may delete it, must leave it for retry-driven reclaim, or must first move it to a terminal state is unstated - and each choice gives a different answer to a later same-key retry.
- **Renewal after expiry is unspecified.** CL-14 permits a worker to extend its lease by a conditional update matching its own `claim_token`, but does not state the lease condition on that update. If renewal is allowed after the lease has expired, a slow worker and a reclaimer can both believe they own the key in the window before the reclaim lands - the duplicate the fencing token exists to prevent.
- **The predicates are described, not stated.** CL-14 section 4 says each conditional update's predicate "includes the expected status, the lease condition where relevant, and the caller's claim_token", without fixing, per operation, which conditions are mandatory. An implementer can satisfy that sentence and still write a renewal with no lease condition.

2. Exact proposed change - purge eligibility

Clarifies v0.3 CL-14 sections 2(f) and 4, and the purge sentence of v0.2 CL-04. No new table, no new column, no new status: `abandoned` remains a derived condition (`in_progress` with an expired lease), exactly as CL-14 defined it. The purge rule is:

Key state	Purge eligibility
`completed`	Purgeable by the maintenance role only, and only after `expires_at` has passed. Before `expires_at` the row is the replay record and must not be deleted. Purging deletes the key row only - never the import batch or run it references, as CL-04 already stated.
`failed` (ordinary)	Purgeable by the maintenance role only after `expires_at` has passed. Before then the row is retained even though a same-hash retry may re-claim it immediately under CL-14: deleting it early would let a different-hash reuse of the key string pass undetected inside the horizon.
`failed` (attempt cap reached)	Same rule: purgeable only after `expires_at`. The terminal-failure response must remain reproducible for the whole horizon, so that a caller retrying a capped key is told to investigate rather than silently re-executing under a fresh claim.
`in_progress`, lease live	**Never purgeable**, at any time, regardless of `expires_at`. A live lease means a worker may be executing a financial operation under this key right now; the row is the only fencing record. The purge predicate in section 4 excludes this state unconditionally rather than comparing clocks.
`in_progress`, lease expired	**Not purgeable directly.** The row must first reach a terminal state by one of the two transitions in section 2 (continued): retry-driven reclaim, which returns it to live `in_progress` under a new claim, or the sweeper transition to `failed`. Only then does the `failed` purge rule apply. An expired claim is a recovery object, not rubbish.

2. Exact proposed change (continued) - recovery of an expired claim, and renewal

How an expired in_progress key is reclaimed or closed. Two routes, and no third. (i) *Retry-driven reclaim:* a same-key, same-hash retry performs the atomic conditional reclaim stated in the predicate table below; if the attempt cap would be exceeded, the same update instead performs the cap transition to terminal `failed`. (ii) *Sweeper transition:* a maintenance process may move an expired claim to `failed` using the sweeper predicate below, clearing the lease, so that work nobody retries does not sit in `in_progress` forever. The sweeper never deletes the row and never executes the operation. Both routes are single conditional updates; PostgreSQL row locking serialises them against each other and against completion, so exactly one transition wins for a given claim.

Renewal after expiry is not permitted. A worker may extend its lease only while the lease is still live. The renewal predicate below therefore includes the condition that `lease_expires_at` is still in the future, in addition to status and token. If the conditional renewal affects zero rows - because the lease expired, or because a reclaim has already rotated the token - the worker no longer owns the key: it must stop, roll back its work transaction if one is open, and must not attempt completion. This holds even if no reclaimer has yet appeared: an expired-but-unreclaimed lease is not ownership, because a reclaimer may already have read the expired state and be between its read and its conditional update; row locking makes the two updates serialise, and the renewal's lease condition is what makes the reclaim win that race deterministically. A worker that needs more time must renew *before* expiry; the lease duration is an operational default (ADAPT, CL-14) chosen with that duty in mind.

2. Exact proposed change (continued) - the conditional-update predicates

The following predicates supersede the descriptive sentence in CL-14 section 4. They are design notation (ADAPT): each operation is exactly one conditional update (or, for purge, one conditional delete by the maintenance role), and *every* condition listed is mandatory. An update that affects zero rows is a defined outcome, stated per operation - never a reason to retry the update without its conditions. Common identity conditions, present in every predicate and not repeated per row: the row's (`merchant_id`, `operation`, `idempotency_key`) match the request, and - where a caller presents one - `request_hash` matches; a hash mismatch is the CL-04 conflict error in every state and authorises no transition.

Operation	Required predicate (all conditions) and effect on success
Lease renewal	`status = 'in_progress'` AND `claim_token = :caller_token` AND `lease_expires_at > now()`. Effect: `lease_expires_at = now() + lease_duration`. Zero rows: ownership lost - stop and roll back; do not complete.
Reclaim (retry-driven)	`status = 'in_progress'` AND `lease_expires_at <= now()` AND `attempt_count < attempt_cap`. Effect: fresh `claim_token`, `lease_expires_at = now() + lease_duration`, `attempt_count = attempt_count + 1`, reclaiming instance recorded. Zero rows: another transition won - re-read the row and follow its state.
Cap transition	`status = 'in_progress'` AND `lease_expires_at <= now()` AND `attempt_count >= attempt_cap`. Effect: `status = 'failed'`, `lease_expires_at = NULL`, token retained for audit. Subsequent same-key retries return the terminal-failure response until purge.
Completion	`status = 'in_progress'` AND `claim_token = :caller_token`. Performed as the last write of the work transaction, as CL-14 requires. Effect: `status = 'completed'`, `completed_at` set, the operation's typed result reference set, `lease_expires_at = NULL`. Zero rows: the claim was reclaimed - the whole work transaction rolls back, discarding its results. Completion carries no lease-time condition of its own: the token is the fence, and if a reclaim had won, the token would no longer match; row locking serialises a completion racing a reclaim so exactly one of them observes the other's committed predicate state.

Operation	Required predicate (all conditions) and effect on success
Worker-reported failure	`status = 'in_progress'` AND `claim_token = :caller_token`. Effect: `status = 'failed'`, `lease_expires_at = NULL`. Zero rows: the worker no longer owns the claim and writes nothing further; the row's state is whatever the winning transition left. A same-hash retry may re-claim a row failed this way immediately, as CL-14 already allowed - failure is a state, not a lock.
Sweeper failure transition	`status = 'in_progress'` AND `lease_expires_at <= now()`. Maintenance role only. Effect: `status = 'failed'`, `lease_expires_at = NULL`. The sweeper holds no token and needs none: expiry is its authority, and the predicate's lease condition is what prevents it from failing a live claim. Zero rows: the claim was renewed, reclaimed or completed first - the correct outcome.
Purge (maintenance delete)	`status IN ('completed','failed')` AND `expires_at <= now()`. Maintenance role only; never a runtime grant. The `status` condition is the operative protection: no `in_progress` row can match it, so no clock comparison is trusted with a live claim's safety. Effect: the key row is deleted; referenced batches and runs are untouched.

Two properties of the table as a whole are part of the change, not commentary on it. First, every transition out of `in_progress` is guarded by either the caller's token or an expired lease - there is no predicate in the lifecycle that can move a live, owned claim. Second, the CL-14 state-to-column CHECK is unchanged and continues to hold after every transition above: `completed` and `failed` rows have `lease_expires_at NULL`; `in_progress` rows have it NOT NULL with all result references NULL. Each predicate's effect maintains exactly that.

3. Affected sections and tables

v0.3 CL-14 section 2(f) (what expiry ends - the purge sentence is superseded by the eligibility table above) and section 2 closing paragraphs; v0.3 CL-14 section 4 (the descriptive predicate sentence is superseded by the predicate table); v0.2 CL-04 (purge permission - read subject to this amendment); Section H (the shared idempotency component gains the purge and sweeper rules); Section I test T-27 (extended, section 5). Table `idempotency_keys`: no schema change - the predicates run entirely on the CL-04 / CL-14 columns. Claim-token fencing, same-transaction completion, atomic reclaim, `attempt_count` and the configured attempt cap (design default 5, ADAPT) are preserved.

4. Constraint or transaction-level enforcement

- **Single conditional updates.** Each lifecycle transition is one statement whose predicate is evaluated against the row under PostgreSQL's row lock; concurrent transitions serialise, and exactly one observes its predicate true. No check-then-update sequence in application code is relied on, as CL-04 already required for the first claim.
- **Purge is a role boundary, not just a predicate.** DELETE on `idempotency_keys` is granted to the maintenance role only. The runtime role cannot purge at all, so a purge bug cannot originate in application code paths.
- **Row-local CHECKs unchanged.** State-to-column consistency remains a CHECK, correctly scoped per CL-17: it inspects one row and validates the outcome of a transition, never the transition's timing.
- **Clock discipline.** All lease and expiry comparisons use the database clock at statement execution (`now()` in the notation above), never a worker-supplied timestamp, so a skewed worker clock cannot extend a lease or accelerate a purge.

5. Updated test scenario

T-27 (extended) - Idempotency recovery and purge. Assertions (h) to (n) are retained, then: (o) a `completed` key inside its replay horizon: purge attempt by the maintenance role deletes nothing; after `expires_at`, purge deletes the key row and the referenced run still resolves, with CL-05 deduplication intact. (p) An `in_progress` key with a live lease and a passed `expires_at`: purge deletes nothing, and the owning worker's completion still succeeds. (q) An `in_progress` key with an expired lease: purge deletes nothing; the sweeper transition moves it to `failed`; only after `expires_at` does purge then delete it. (r) Renewal: renewal before expiry succeeds and extends the lease; renewal attempted after expiry - with no reclaimer yet - affects zero rows, and the worker's subsequent completion attempt also affects zero rows once a reclaim has rotated the token. (s) Race the sweeper against a renewal at the expiry boundary: exactly one transition wins, and the row is never simultaneously failed and leased. (t) A capped key: purge before `expires_at` deletes nothing and the terminal-failure response is still returned; after `expires_at` the row is purgeable.

6. Status

Design-resolved; implementation verification outstanding. Purge eligibility, expired-claim recovery, the renewal boundary and every lifecycle predicate are now stated exactly, on the existing CL-04 / CL-14 schema, with no new table and no weakened fence. The predicates are design notation: their race behaviour must be demonstrated under concurrent load in the B-3 environment before this finding is treated as verified.

CL-20 - CL-15 shared-user anonymisation serialisation

1. Original problem

v0.3 CL-15 section 4 specified the concurrency mechanism for global user anonymisation in one sentence: the Stage 2 maintenance transaction *takes the relevant membership rows under lock so a concurrent invitation or reactivation of the same user in another workspace cannot race the anonymisation*. That mechanism has a hole of a well-known shape: **you cannot lock a row that does not exist yet.**

- **Phantom membership.** Stage 2 locks the user's existing `merchant_memberships` rows, finds none active or invited, and proceeds. Concurrently, a workspace the user has never belonged to creates a brand-new membership - or an invitation - for that user. There was no row to lock, the eligibility recheck has already passed, and the transaction anonymises a `users` row that now belongs to a person with a live membership elsewhere: precisely the outcome CL-15's shared-user rule forbids.
- **The racing transactions were never told to participate.** Even for rows that do exist, row locking only serialises writers who contend for the same row in a conflicting mode. CL-15 imposed a locking duty on Stage 2 alone; membership creation, invitation, reactivation and status change were given no protocol at all, so nothing required them to wait for, or be waited for by, an anonymisation in progress.
- **Sessions were outside the mechanism.** CL-15's eligibility question includes whether the user still has a live session, but session creation was not part of the locking story either: a sign-in completing during Stage 2 could create a live session for a user whose identity fields are being erased in the same instant.

The tombstone design itself is not in question: the merchant row is anonymised in place, `merchant_id` never moves on any financial record, and referential integrity is never at risk. What is missing is a serialisation point that exists before the membership row does.

2. Exact proposed change - one per-user lock, taken by everyone

Clarifies and strengthens v0.3 CL-15 sections 2 and 4. The serialisation point is a **transaction-scoped advisory lock keyed by the user's ID** (design notation, ADAPT: `pg_advisory_xact_lock` over a stable 64-bit derivation from the `users.id` UUID, in a lock namespace reserved for this protocol so its keys cannot collide with the CL-13 candidate locks or any other advisory-lock user). The choice follows the reasoning CL-13 already established: an advisory lock needs no table privilege, modifies no row, is held only to commit or rollback, and - being transaction-scoped - cannot leak through a pooled connection. Unlike a row lock, it exists independently of whether any membership row has been created yet, which is the entire point.

The protocol is uniform. Every transaction in the following list must acquire the per-user lock for the affected `user_id` before it reads or writes anything the lock protects, and must hold it to commit. A transaction that does not take the lock is non-conforming by definition, in the same way CL-13 made the candidate lock a design invariant of decision inserts and CL-11 made tenant context a design invariant of tenant queries:

- membership creation (a new `merchant_memberships` row for an existing user);
- invitation creation where the invitee is an existing user, and invitation acceptance in all cases - acceptance is the moment an email-only invitation becomes a membership of a known user, so the race is closed at that point even where creation could not lock;
- membership reactivation;
- any membership status change that affects anonymisation eligibility - in practice, every status change, since a transition's effect on eligibility should never have to be argued case by case;
- session creation for the user (see section 2, continued);
- workspace Stage 2 anonymisation, for each user whose `users` row it may anonymise.

2. Exact proposed change (continued) - Stage 2 under the lock, and lock ordering

Stage 2 procedure. The Stage 2 maintenance transaction proceeds per workspace in this order: (1) determine the candidate user set - users whose memberships in this workspace are ended and who are therefore candidates for global anonymisation; (2) acquire the per-user lock for each candidate, in ascending order of the lock key (see ordering below); (3) with all locks held, *recheck* eligibility from current data - the candidate set and any earlier eligibility reading are stale by construction and are used for nothing but choosing whom to lock; (4) anonymise in place exactly those `users` rows that pass the recheck, and record in the audit row, per user, whether anonymisation happened or was deferred and why. A user who fails the recheck is simply not anonymised by this workspace's Stage 2; nothing about the workspace's own tombstoning changes.

The recheck, stated exactly. A `users` row is eligible for anonymisation only if, under the lock, all three hold: no `merchant_memberships` row for that user in *any* workspace has a status that counts as live under CL-15 - active or invited; no live session exists for the user - no `user_sessions` row that is unrevoked and unexpired; and the user is not the subject of a pending invitation acceptance in flight - which the protocol makes impossible to overlook, because acceptance itself takes the same lock and therefore either committed before the recheck (and is visible to it) or waits until Stage 2 commits. Because every membership and session mutation takes the same lock, the recheck is not a hopeful snapshot: no conforming transaction can change any of the three conditions while the locks are held.

Lock ordering and deadlock discipline. Advisory locks deadlock like any other locks if taken in inconsistent orders. The protocol therefore fixes a global order: where a transaction needs more than one per-user lock (Stage 2, in practice), it acquires them in ascending lock-key order and acquires no other advisory lock afterwards. A transaction that already holds a CL-13 candidate lock never acquires a per-user lock - the two protocols govern disjoint operations, and the design states the disjointness so an implementation does not invent a combined path. Ordinary row locks taken after the advisory lock follow the tables' existing access patterns; the B-3 verification environment must include a deadlock test for concurrent Stage 2 and invitation traffic (section 5).

What is preserved. The tombstone merchant approach is unchanged: the `merchants` row is anonymised in place, `anonymised_at` is set, no `merchant_id` on any financial record is ever rewritten, and financial rows remain byte-identical. A user with an active or invited membership in any other workspace is never anonymised - now as a serialised guarantee rather than a checked intention. Stage 2 remains a maintenance transaction under the migrator role only; the runtime role gains no grant. **OD-16 and OD-17 remain open**: this finding strengthens mechanics only and decides neither retention periods nor the treatment of personal data inside append-only raw records.

3. Affected sections and tables

v0.3 CL-15 section 2 (shared-user rule - the eligibility conditions gain the live-session condition in the recheck form above) and section 4 (the membership-row locking sentence is superseded by the lock protocol); v0.1 Section C, tables `merchant_memberships` and `user_sessions` (access protocol only - no schema change); Section G (Stage 2 procedure); Section I tests T-25 (extended) and T-30 (extended). No table, column or constraint changes anywhere in this finding.

4. Constraint or transaction-level enforcement

- **Transaction-level protocol, honestly labelled.** Cross-table eligibility - memberships anywhere, sessions anywhere - cannot be a CHECK, per CL-17, and no deferred constraint trigger is specified for it here: a trigger on `merchant_memberships` cannot prevent the phantom-insert race either, because the anonymising write is on `users`, in a different transaction. The enforcement is the lock protocol itself: one shared component through which every listed mutation passes, stated as a design invariant in the CL-11 / CL-13 sense, with conformance testable by the concurrency scenarios in section 5.
- **Stage 2 fails closed.** If any per-user lock cannot be acquired within the maintenance transaction's bounded wait, Stage 2 aborts for that workspace rather than proceeding on a partial lock set. A deferred anonymisation is a safe outcome; an unserialised one is not.
- **No privilege changes.** Advisory locks require no table privilege and confer none. The migrator-only character of Stage 2, the runtime role's grants, and the closed-workspace write policies of CL-12 are all unchanged.
- **Audit.** The Stage 2 audit record of CL-15 is extended in content, not in kind: it records per candidate user the recheck outcome under lock (anonymised, or deferred with the blocking condition named by category - live membership, live session - never personal values).

5. Updated test scenario

T-30 (extended) - Shared user across workspaces, under concurrency. The v0.3 assertions are retained, then: (h) *Phantom membership race:* begin Stage 2 for workspace A; concurrently create a first-ever membership for candidate user U in unrelated workspace C. Expected: the two transactions serialise on U's lock - either the membership commits first and Stage 2's recheck finds it and does not anonymise U, or Stage 2 commits first and the membership creation then proceeds against an already-anonymised user only if the application's own rules permit inviting an anonymised identity (a product rule this design does not invent; the database outcome asserted is that the two outcomes never interleave into anonymising a user with a committed live membership). (i) *Invitation race:* repeat (h) with an invitation to an existing user, and separately with an email-only invitation whose acceptance runs concurrently with Stage 2: acceptance serialises on the same lock, with the same either/or outcome. (j) *Session race:* a sign-in creating a session for U concurrently with Stage 2: either the session exists at recheck and anonymisation is deferred, or anonymisation commits first and the session creation observes the anonymised row. (k) *Reactivation race:* a reactivation of U's ended membership in workspace B concurrent with Stage 2 for A serialises identically. (l) Run (h) to (k) under repeated concurrent load and assert no deadlock persists unresolved: PostgreSQL deadlock detection may abort a participant, which the protocols treat as a retryable failure, never as a silent pass.

T-25 (extended). Add: Stage 2 executed while holding only a subset of candidate locks - simulated by a non-conforming test transaction - is not a state the conforming protocol can produce; the assertion recorded is that the Stage 2 procedure as specified acquires the full sorted lock set before its recheck, verified by the concurrency outcomes of T-30(h) to (l).

6. Status

Partially resolved - mechanics design-resolved; policy still open. The phantom-membership hole is closed at design level by a single per-user serialisation point that every relevant mutation must share, with Stage 2 rechecking under the lock. OD-16 (grace window, retention periods, erasure-versus-retention conflict rule) and OD-17 (third-party personal data in append-only raw records) remain open for Bangladesh legal and policy review, exactly as before. The lock protocol's behaviour under real concurrency is unverified until the B-3 environment exists.

CL-21 - CL-16 same-batch input integrity and current variance validity

1. Original problem

v0.3 CL-16 built bank variance history on two append-only tables: `settlement_variance_calculations` (one row per computation for one settlement batch) and `settlement_variance_calculation_inputs` (the typed receipt and allocation references that computation used). Its tenant guarantee is sound - every reference is a composite (`merchant_id`, `id`) foreign key, so a calculation cannot list another workspace's receipt or allocation. But tenant equality is the only relationship the keys express, and it is not enough:

- **Same merchant, wrong batch, is representable.** An input row for a calculation on batch B1 can reference a receipt linked to batch B2 of the same merchant, or an allocation belonging to B2. Every foreign key is satisfied. The calculation's snapshot totals would then describe one batch while its input set describes another, and the deferred trigger of CL-16 section 4 - which checks snapshots against the listed inputs - would faithfully verify the wrong set.
- **The gap is structural, not hypothetical.** The two reference types reach the batch by different paths in the existing schema: an allocation carries its own `settlement_batch_id` (NOT NULL, v0.1 Section C), while a receipt's `settlement_batch_id` is nullable - a receipt may legitimately be recorded before its batch is identified, and such a receipt cannot contribute to any variance. Nothing in the inputs table connects either path to the parent calculation's batch.
- **Current validity ignores set changes with equal totals.** CL-16 derives the current variance from the latest calculation *if its preconditions still hold*, tested by comparing snapshot values: expected payout, net allocated total, receipt completeness. A reversal followed by a re-allocation of the identical amount - new allocation rows, same net - leaves every compared number unchanged. The CL-16 rule therefore continues to present the old calculation as current even though the input set it was computed from no longer exists. For an audit record, that is the wrong answer: the figure being shown was not computed from the allocations now in force, and after a dispute the trail would claim otherwise.

2. Exact proposed change - batch equality by composite foreign key

Strengthens v0.3 CL-16 sections 2 and 4 within the two existing tables. No third table is added; the existing schema *can* support the requirement once the inputs table carries the batch it belongs to, so the no-new-table condition in the commissioning brief is met by keys, not storage. Three supporting unique keys and one column are added (all ADAPT as design notation):

Addition	Purpose
`settlement_variance_calculations`: UNIQUE (`merchant_id`, `settlement_batch_id`, `id`)	Lets children reference a calculation *as a calculation of a specific batch*, not merely as a calculation of the merchant.
`settlement_variance_calculation_inputs.settlement_batch_id`, UUID NOT NULL	The batch the input is asserted to belong to, copied from the parent calculation in the computing transaction. Consistency with the parent is enforced by the composite FK below, not by care.
Inputs: composite FK (`merchant_id`, `settlement_batch_id`, `calculation_id`) references `settlement_variance_calculations` (`merchant_id`, `settlement_batch_id`, `id`)	An input's batch is its calculation's batch, by construction. An input row that disagrees with its parent about the batch cannot be inserted.
`bank_receipts`: UNIQUE (`merchant_id`, `settlement_batch_id`, `id`); `settlement_allocations`: UNIQUE (`merchant_id`, `settlement_batch_id`, `id`)	Referenceable batch-scoped identities for receipts and allocations. Both are superkeys over an already-unique `id`, so they constrain nothing that exists - they exist so the foreign keys below can be expressed.

With the batch on the input row, the two reference foreign keys are restated in batch-scoped form, replacing the CL-16 (`merchant_id`, `id`) forms:

- Receipt inputs: composite FK (`merchant_id`, `settlement_batch_id`, `bank_receipt_id`) references `bank_receipts` (`merchant_id`, `settlement_batch_id`, `id`).
- Allocation inputs: composite FK (`merchant_id`, `settlement_batch_id`, `settlement_allocation_id`) references `settlement_allocations` (`merchant_id`, `settlement_batch_id`, `id`).

The CL-16 exactly-one-reference rule (the row-local XOR CHECK over the two reference columns, keyed by `input_kind`) is what makes these keys airtight: for a receipt input, `bank_receipt_id` is NOT NULL, so every component of its foreign key is non-null and the key is fully checked under the default MATCH SIMPLE semantics - the referenced receipt's batch must equal the input's batch, which the parent FK has already made equal to the calculation's batch. The inapplicable key (the allocation key on a receipt row) has a NULL component and is vacuously satisfied, which is correct: it asserts nothing about a reference the row does not make. Two honest consequences are stated so an implementer is not surprised by them. First, a receipt with `settlement_batch_id` NULL can never be a variance input - the key cannot match - which restates the v0.1 rule that an unlinked receipt contributes to no variance. Second, because a referenced key cannot be updated while references exist, re-linking a receipt to a different batch after it has served as a calculation input is blocked by the database; the correction path is a new calculation after re-allocation, never a quiet edit of history's inputs.

2. Exact proposed change (continued) - current validity is input-set equality

Supersedes the *still-holds* comparison in v0.3 CL-16 section 2. The current variance for a batch is the `bank_variance_poisha` of the batch's highest-`calculation_seq` calculation **if and only if that calculation's recorded input set is exactly the batch's current relevant input set**, and its CL-16 preconditions still hold. In every other case the current variance is NULL. The comparison is set equality over identifiers, performed per kind:

- **Receipt set.** The calculation's receipt inputs (by `bank_receipt_id`) must equal, as a set, the receipts currently linked to the batch - every `bank_receipts` row whose `settlement_batch_id` is this batch. A receipt linked after the calculation, or unlinked from it, breaks equality even if no amount moved.
- **Allocation set.** The calculation's allocation inputs (by `settlement_allocation_id`) must equal, as a set, all allocation rows currently recorded for the batch - including reversal rows, which are allocations in their own right under CL-10 and were listed as inputs in their own right under CL-16. A reversal, or a reversal followed by an equal re-allocation, breaks equality by construction: the new rows have identifiers the recorded set does not contain.
- **Preconditions retained.** Every currently linked receipt must still be explicitly complete, and the CL-16 snapshot equalities (expected payout; net allocated total) must still hold. Set equality is added to the CL-16 tests, not substituted for them.

The rule is deliberately stricter than numerical equality, and the reason is audit traceability, as the commissioning brief directed: a presented variance asserts *this figure was computed from these receipts and these allocations*. When the *these* changes, the assertion is false even if the arithmetic coincidentally reproduces the same number. The remedy is cheap and already in the design: a new calculation with the next `calculation_seq`, snapshotting the new set - which will record the same variance value as a new, truthful record rather than letting the old record impersonate it. Nothing is lost: the superseded calculation remains queryable in sequence, exactly as CL-16 specified.

3. Affected sections and tables

v0.3 CL-16 section 2 (inputs table: one column added, two foreign keys restated in batch-scoped form, one parent FK added; current-variance derivation: the still-holds test is superseded by input-set equality plus the retained preconditions) and section 4 (enforcement); Section B ERD settlement sub-diagram (key annotations only); Section C tables `bank_receipts` and `settlement_allocations` (one supporting unique key each - no column or rule changes); Section D.10; Section I tests T-28 (extended) and T-31 (new). Preserved without change: exactly two variance tables, append-only calculation and input records, `calculation_seq` sequencing, historical calculations after reversal, the derived `bank_variance_poisha` read surface, the absence of any stored mutable variance column on `settlement_batches`, all financial formulas, and CL-10 allocation and reversal semantics - CL-10's removal of the mutable `is_active` flag means the allocation set in section 2 is simply all allocation rows for the batch, with no active-flag filtering to argue about.

4. Constraint or transaction-level enforcement

- **Batch equality is foreign-key enforcement** - immediate, row-by-row, role-independent - not a trigger and not a transaction convention. A cross-batch input fails at insert, in the computing transaction, before any question of commit arises. The deferred constraint trigger retained from CL-16 section 4 keeps its narrower job: verifying that a calculation's snapshots agree with its (now provably same-batch) listed input set and preconditions at commit.
- **Current validity is a derivation rule**, evaluated at read time over current sets, in the same spirit as CL-16's original derivation: it stores nothing, updates nothing, and cannot disagree with a stored flag because there is none - CL-16's rejection of an `is_current` flag stands.
- **The computing transaction is unchanged in shape** - batch row lock, precondition verification, atomic insert of calculation plus its complete input set - except that the input insert now also carries the batch, and the foreign keys verify the carry.
- **Execution context.** The retained deferred trigger follows the CL-22 pattern of this amendment for its visibility under FORCE RLS and its fail-closed behaviour.

5. Updated test scenario

T-31 (new) - Same-batch inputs and input-set validity. (a) Attempt a calculation input referencing a same-merchant receipt linked to a *different* batch: rejected at insert by the batch-scoped composite FK. (b) Attempt an input referencing a same-merchant allocation of a different batch: rejected identically. (c) Attempt an input referencing a receipt whose `settlement_batch_id` is NULL: rejected - no key match exists. (d) *Changed set, unchanged net:* complete a batch and compute calculation seq 1; reverse one allocation and re-allocate the identical amount as a new allocation row, restoring the same net total; expected - the recorded input set no longer equals the current allocation set, current variance is NULL, and no new calculation exists yet. (e) Run a fresh calculation after the re-allocation: seq 2 exists, its input set includes the reversal row and the new allocation, current variance equals seq 2's value - numerically identical to seq 1's, and asserted as a *different record* - and seq 1 remains queryable and unchanged. (f) After all of the above, attempt UPDATE and DELETE on both variance tables as the runtime role: denied or affecting zero rows; every historical calculation and input row is byte-identical to its state at creation.

T-28 (extended). Add: an input row whose `settlement_batch_id` disagrees with its parent calculation's batch is rejected by the parent composite FK; and linking an additional receipt to the batch after seq 1 - before completing it - makes current variance NULL by set inequality alone.

6. Status

Design-resolved; implementation verification outstanding. Same-batch integrity is now carried by composite foreign keys the schema can genuinely support, and current variance validity is defined by input-set equality, so a changed set with an unchanged total invalidates the presented figure until a new calculation records it truthfully. MATCH SIMPLE behaviour over the XOR reference columns is standard PostgreSQL semantics, but the assembled key structure must still be exercised in the B-3 environment before this finding is treated as verified.

CL-22 - CL-17 trigger execution context under FORCE RLS

1. Original problem

v0.3 CL-17 assigned every cross-table invariant in the design to a deferred constraint trigger, and added a short note on trigger functions and RLS: the functions execute with the privileges of the function owner (the migrator role), FORCE RLS subjects even the owner to policies, and therefore a verification function must scope every query by the `merchant_id` of the rows that fired it rather than relying on the caller's tenant context. The note names the right discipline but leaves the mechanism by which it can actually work unspecified - and in this area an unspecified mechanism defaults to one of two failure modes, both unacceptable:

- **Silent under-reading (the dangerous one).** If the function's queries are filtered by policies keyed to a tenant context that is absent, stale or set for a different merchant - a maintenance transaction, a pooled connection reused across tenants, a caller that set context for merchant A while writing merchant B's rows - the function sees a subset of the rows it must validate, or none. A validation that inspects an empty set finds no inconsistency. The transaction commits. The design's strongest guarantees (the CL-01 status derivation, CL-13 chain shape, CL-16 snapshot agreement) would then hold exactly when nothing went wrong with the context, and silently not hold when something did - the precise inversion of a safety mechanism.
- **The over-broad escape.** The obvious fix - make the function owner a role that bypasses RLS - hands the validation path a general-purpose bypass. If that role or its functions are reachable in any other way, tenant isolation has a standing hole in it, and the design has traded a correctness risk for a security one.

Neither v0.1 Section E nor CL-17 states: who owns these functions and with what execution mode; which policies admit the function's reads and on what condition; where the `merchant_id` it validates comes from; or what the function does when it cannot see the rows that fired it. This finding specifies all four, once, as a pattern every deferred constraint trigger in the design must follow - the CL-01 status trigger, the CL-13 chain trigger (as scoped by CL-18 of this amendment), and the CL-16 variance trigger.

2. Exact proposed change - the validation trigger context pattern

Adds a subsection to v0.1 Section E and completes the trigger note in v0.3 CL-17 section 2. Six rules, all mandatory for every deferred constraint-trigger function in the design (design notation throughout, ADAPT):

Rule 1 - Owner and execution mode. Each validation function is owned by the migrator role - the same non-superuser role that owns the schema objects - and is declared SECURITY DEFINER, so it executes with its owner's privileges regardless of which role's write fired the trigger. It is never owned by a superuser, and the owner role does not carry the BYPASSRLS attribute. SECURITY DEFINER here is a scoping device, not an elevation: it fixes *whose* policies govern the function's reads (the owner's) so that behaviour does not vary with the caller, and Rules 3 and 4 then constrain what those reads can reach.

Rule 2 - merchant_id comes from the firing rows, never from context. The function takes the tenant it validates from the row that fired the trigger - the inserted or updated row's own `merchant_id` column, present on every tenant-owned table by the v0.1 convention. The application tenant-context setting is not consulted for this purpose at all: not as the source of the merchant, and not as a fallback when the column is somehow unavailable (a firing row without a `merchant_id` is a schema defect that must raise, Rule 5). Where a transaction fires the trigger for several merchants - which conforming application transactions never do, but a maintenance transaction could - the function validates each firing row's merchant separately, deriving each scope from its own row.

Rule 3 - A dedicated validation policy admits exactly one merchant's rows, to the owner role only. Under FORCE RLS the owner's reads are policy-filtered like everyone else's, so each table a validation function reads carries one additional permissive SELECT policy, applicable *to the owner role only*, whose condition is that the row's `merchant_id` equals a transaction-local validation setting. Before querying, the function sets that setting - from the firing row's `merchant_id`, per Rule 2 - using a transaction-local set operation, so the value dies with the transaction and cannot leak through a pooled connection (the same transaction-locality CL-11 already requires of tenant context). The ordinary runtime policies are untouched, and the runtime role is not a member of, and cannot assume, the owner role: the validation policy's condition is simply never evaluated in the runtime role's favour, so a runtime session that sets the same setting gains no row it could not already see. This is the narrow alternative to a bypass: one role, one merchant at a time, one transaction at a time, read-only.

Rule 4 - Every query is explicitly tenant-scoped anyway. Policy admission is the second line, not the first. Every query inside a validation function carries an explicit `merchant_id` predicate equal to the firing row's merchant, on every table it touches. A missing or wrong policy then produces a visible failure (Rule 5) rather than a widened read, and a missing predicate is a review defect under the CL-17 drafting rule, extended by this amendment to trigger bodies: a reviewer must be able to point at the tenant predicate in every query the function makes.

Rule 5 - Fail closed: invisibility is an error, never a pass. Before validating, the function confirms it can see its anchor: the firing row itself, re-read through its own scoped queries. If the anchor read returns no row - because the validation setting was not established, the policy is missing, or the scope was derived wrongly - the function raises an exception and the transaction aborts. The same rule applies per merchant in a multi-merchant firing. Under no circumstances does a validation function treat an empty read as a consistent state. This rule converts every configuration error in Rules 1 to 4 from a silent hole into a loud, transaction-blocking failure - which is why the pattern can be recommended before it has been run: its failure mode is refusal, not leakage.

Rule 6 - No privilege reaches the runtime role. The pattern grants the application role nothing: no new table grant, no role membership, and no callable surface. Validation functions return the trigger type, are invocable only in trigger context, and EXECUTE on them is in any case not granted to the runtime role (defence in depth, stated so an implementer adds the revocation deliberately rather than relying on the return-type behaviour alone). The runtime role cannot set the owner role's policy in its favour (Rule 3), cannot call the function directly, and cannot suppress the trigger (CL-18). What the runtime role experiences is unchanged from CL-17: it writes rows it is permitted to write, and inconsistent transactions fail at commit.

> **Rejected alternatives, recorded.** A BYPASSRLS validation owner and a superuser-owned function are both rejected: each is a general-purpose bypass whose reach is not limited to one merchant, one transaction or read-only access, and neither can satisfy Rule 5's spirit - both would read everything and therefore never notice a scoping error. Relying on the caller's tenant context (the position CL-17's note warned against without replacing) is rejected for the under-reading failure in section 1. Performing validation only in application code is rejected as it was in CL-17: a financial invariant with no database backstop is a convention, not a constraint.

3. Affected sections and tables

v0.1 Section E (new subsection: the validation trigger context pattern; the role and policy specifications gain the owner-only validation SELECT policy per validated table); v0.3 CL-17 section 2 (the trigger-and-RLS note is superseded by Rules 1 to 6); v0.3 CL-13 section 4 and CL-16 section 4, and CL-18 and CL-21 of this amendment (their deferred triggers are declared subject to the pattern); Section I test T-32 (new), with T-29 extended in one assertion (section 5). No table or column changes. Tenant isolation is not weakened: the only new policy is SELECT-only, owner-role-only, single-merchant and transaction-scoped.

4. Constraint or transaction-level enforcement

- **Pattern conformance is a review gate.** Every deferred constraint trigger introduced anywhere in v0.1 to v0.4 is listed in the CL-17 mechanism map; implementation review checks each function against Rules 1 to 6 in order, and a function that cannot demonstrate Rule 5's anchor check does not pass review.
- **Deferred timing is what makes the pattern safe.** Because the triggers are DEFERRABLE INITIALLY DEFERRED, the function runs at commit inside the firing transaction: the transaction-local validation setting is still alive, the firing rows are visible to the transaction, and an exception aborts the whole transaction atomically - no partial validation state can persist.
- **Pooling safety follows from transaction-locality twice over.** Both the application tenant context (CL-11) and the validation setting are transaction-local, so a pooled connection cannot carry either into another tenant's work - and Rule 2 means the validation outcome would not depend on the carried value even if one survived.

5. Updated test scenario

T-32 (new) - Validation triggers under the production role and RLS configuration. Executed as the runtime role, against a database with RLS ENABLE + FORCE and the production policy set - explicitly not as the migrator or a superuser, since a privileged test session would mask every defect this scenario exists to catch. (a) Commit a result whose findings contradict its stored overall status under the CL-01 derivation, with correct tenant context set: rejected at commit by the deferred trigger. (b) Repeat with the tenant context deliberately set to a *different* merchant than the rows being written (the write itself proceeds under whatever policy governs it; the assertion is on validation): the trigger derives its scope from the firing rows, still detects the inconsistency, and rejects at commit. (c) Repeat with tenant context entirely unset in the transaction: the transaction fails - by policy denial or by the function's fail-closed raise - and under no outcome does it commit; the test records which failure occurred, because both are acceptable and a commit is not. (d) A fully consistent result-and-findings transaction commits normally, proving the pattern does not over-block. (e) As the runtime role, set the validation setting directly and attempt to read another merchant's rows, and attempt to invoke a validation function directly: no additional row is visible. (f) Disable the validation policy on one table in the test database (simulating a configuration error): the next inconsistent commit attempt fails closed under Rule 5 rather than passing on an empty read.

T-29 (extended). Add one assertion to part (b): the commit-time rejection is demonstrated under the T-32 configuration - runtime role, FORCE RLS active - and not only under a privileged session.

6. Status

Design-resolved; implementation verification outstanding. The execution context CL-17 assumed is now specified: owner, execution mode, merchant derivation, scoped visibility, explicit predicates and a fail-closed rule, with no additional privilege for the runtime role and no general-purpose bypass introduced. Of all five findings in this amendment, this is the one whose behaviour is hardest to prove on paper - SECURITY DEFINER interaction with FORCE RLS policies is exactly the kind of mechanism the B-3 verification environment exists to test, and T-32 is written to be run there before the pattern is relied on.

Consolidated test changes

Section I of v0.1, as amended by v0.2 and v0.3, stands except as listed here. This table is the complete delta from this amendment. No test is claimed to have passed: these are design-stage scenarios for the future M1 test suite, and the concurrency and RLS scenarios among them can only be executed once the B-3 verification environment exists.

Test	Change	Source
T-25	Extended: Stage 2 conformance assertion - the full sorted per-user lock set is acquired before the eligibility recheck	CL-20
T-26	Part (e) revised: the malformed history is a labelled legacy fixture (load-before-trigger or test-only replica-mode load), with all detection assertions run under the fully active trigger. New part (h): the active trigger rejects a newly created malformation for the runtime and migrator roles alike, including a further root added to a legacy-malformed candidate	CL-18
T-27	Extended with parts (o) to (t): purge inside and outside the replay horizon, purge refusal for live and expired in_progress keys, sweeper transition, renewal before and after expiry, sweeper-versus-renewal race, capped-key purge timing	CL-19
T-28	Extended: input batch disagreeing with its parent calculation rejected by the parent composite FK; later receipt linkage invalidates current variance by set inequality alone	CL-21
T-29	Extended: the commit-time rejection of part (b) is demonstrated under the T-32 configuration (runtime role, FORCE RLS), not only under a privileged session	CL-22
T-30	Extended with parts (h) to (l): phantom membership, invitation and invitation-acceptance races, session race, reactivation race, and deadlock behaviour under concurrent Stage 2 and membership traffic	CL-20
T-31	New: same-batch input integrity - cross-batch receipt and allocation inputs rejected, NULL-batch receipt rejected, changed input set with unchanged net invalidates current variance, valid new calculation after re-allocation, historical records unchanged	CL-21
T-32	New: validation trigger execution context - commit-time detection under the production runtime role and FORCE RLS, wrong-context and missing-context runs, direct-invocation and setting-abuse attempts by the runtime role, fail-closed behaviour when the validation policy is missing	CL-22

Invariant traceability delta (Appendix 1)

- Row "Match decisions form one linear chain per candidate" - enforcement now reads: advisory-lock serialisation, partial unique successor index, same-candidate composite self-FK, and a deferred constraint trigger scoped to malformation the committing transaction introduces (CL-18); pre-existing malformation is governed by the deterministic derivation, read-only. Tests T-26(e) revised, T-26(h).
- Row "A claimed idempotency key always reaches a defined end state" - now also covers purge and lease lifecycle: purge only for terminal keys after `expires_at`; `in_progress` keys reach an end state only by token-guarded or expiry-guarded conditional transitions (CL-19). Tests T-27(o) to (t).
- Row "Anonymisation changes identity, never keys" - enforcement now includes the per-user advisory-lock protocol shared by every membership and session mutation and by Stage 2, with eligibility rechecked under the lock (CL-20). Tests T-25, T-30(h) to (l).
- Row "Bank variance NULL until receipt complete and batch fully allocated" - now also covers: every calculation input belongs to the calculation's own batch, by composite foreign key; and the current variance is derived only while the latest calculation's recorded input set equals the current receipt and allocation sets (CL-21). Tests T-28, T-31.
- New row: cross-table validation reads what it validates - deferred constraint-trigger functions are migrator-owned, SECURITY DEFINER, derive `merchant_id` from the firing rows, read under an owner-only single-merchant validation policy with explicit tenant predicates on every query, and raise rather than pass when their anchor rows are invisible (CL-22). Test T-32.

Change log - v0.3 to v0.4 (this amendment)

Item	Type	Change
CL-18	Clarifies (corrects v0.3 test)	CL-13's deferred constraint trigger scoped to the delta: it rejects malformation the committing transaction introduces, for every role, and neither repairs nor is blocked by pre-existing malformation. Case A (created) and Case B (pre-existing) defined. T-26(e) revised to a labelled legacy-data fixture - load-before-trigger, or a test-only replica-mode load by the migrator role in a disposable database - with detection asserted under the fully active trigger; T-26(h) added. No weakening of the production trigger, the lock, the indexes or the derivation.
CL-19	Clarifies + Adds	CL-14's purge sentence replaced by a state-by-state eligibility rule: completed and failed keys purgeable by the maintenance role only after `expires_at`; `in_progress` keys never purgeable - a live lease unconditionally, an expired lease until it first reaches a terminal state by reclaim or the sweeper transition. Renewal after lease expiry prohibited. Exact mandatory predicates stated for renewal, reclaim, cap transition, completion, worker-reported failure, sweeper failure and purge. No schema change. T-27 extended (o) to (t).
CL-20	Clarifies + Strengthens	CL-15's membership-row locking superseded by a uniform per-user transaction-scoped advisory-lock protocol covering membership creation, invitation creation and acceptance, reactivation, eligibility-relevant status changes, session creation and Stage 2. Stage 2 acquires candidate locks in ascending key order and rechecks global membership and live-session eligibility under the lock; failure to acquire the full set aborts Stage 2. Tombstone approach and all key preservation unchanged. OD-16 and OD-17 remain open. T-25 and T-30 extended.
CL-21	Supersedes + Adds	CL-16 input integrity strengthened without a new table: `settlement_batch_id` added to `settlement_variance_calculation_inputs`; supporting unique keys added on calculations, `bank_receipts` and `settlement_allocations`; input foreign keys restated in batch-scoped composite form so a same-merchant input from another batch - or an unlinked receipt - is rejected at insert. Current-variance validity superseded: derivation now requires exact input-set equality (receipt set and allocation/reversal set) in addition to the retained CL-16 preconditions, so a changed set with an identical net yields NULL until a new calculation. T-28 extended; T-31 added.
CL-22	Clarifies + Adds	CL-17's trigger-and-RLS note superseded by a six-rule validation trigger context pattern in Section E: migrator-owned SECURITY DEFINER functions without BYPASSRLS; `merchant_id` derived from the firing rows, never from caller context; an owner-only, single-merchant, transaction-local validation SELECT policy; explicit tenant predicates on every query; a fail-closed anchor check that raises instead of validating an empty set; and no new privilege or callable surface for the runtime role. BYPASSRLS, superuser ownership and context-reliant validation rejected by name. T-29 extended; T-32 added.

Types follow v0.2 and v0.3 usage: Supersedes - the earlier text named is replaced for that item only. Clarifies - the earlier text was ambiguous, incomplete or internally inconsistent; one reading is fixed and the other rejected explicitly. Adds - new material with no direct counterpart. Strengthens - an existing mechanism is replaced by a strictly stronger one for a demonstrated reason.

Remaining blockers and open decisions

Blocking any M1 implementation (unchanged from v0.1, v0.2 and v0.3)

- **B-1. Market validation:** Day 7 and Day 14 gates not yet run; no interview evidence exists.
- **B-2. No merchant-confirmed courier evidence** for any of Steadfast, Pathao, Paperfly, RedX or eCourier - no rule version can yet be populated without inventing contract semantics, which is prohibited.
- **B-3. Docker + PostgreSQL verification environment not established.** No design SQL or mechanism in v0.1 to v0.4 has been executed. This amendment adds five mechanisms to that unverified list, and names them so they are tested first when the environment exists: trigger delta identification (CL-18), the conditional-update race behaviour (CL-19), advisory-lock ordering under concurrent membership and Stage 2 traffic (CL-20), the batch-scoped composite keys over the XOR input references (CL-21), and SECURITY DEFINER trigger visibility under FORCE RLS (CL-22, exercised by T-32).
- **B-4. Explicit production-readiness approval not given.**

Open decisions carried forward

- **OD-12** (CL-02, v0.2) - whether any real courier statement reports a negative net payout. Open until redacted merchant statements answer it. Untouched by this amendment.
- **OD-16** (CL-12, v0.2) - workspace grace-window length, Bangladesh financial retention period, and the erasure-versus-retention conflict rule. Requires policy and legal review. CL-20 supplies serialisation mechanics; the periods and the conflict rule remain undecided. Blocks production launch, not design review.
- **OD-17** (CL-15, v0.3) - treatment of third-party personal data held inside append-only raw records when a workspace is anonymised. Interim position unchanged: sealed retention, excluded from anonymised exports, no runtime-role access. Requires the same Bangladesh legal review as OD-16. Blocks production launch, not design review.
- **OD-11** (v0.1, narrowed by CL-17) - guaranteeing at least one active owner per workspace: mechanism choice between a locked membership transaction and a deferred constraint trigger remains open. Noted here because CL-20's lock protocol is a candidate component for the first option; this amendment does not make that choice.
- **CL-11 deployment check** (v0.2) - the tenant-context trust boundary is specified; verifying a real deployment honours it remains a production-readiness task. CL-18 and CL-22 add two items to that check: no production path may suppress a constraint trigger for fixture purposes, and the validation policy set must match the CL-22 pattern in the deployed database.
- All other v0.1 open decisions OD-01 to OD-15 stand as written, except OD-11 and OD-12 as noted above.

Is the database design ready for schema freeze?

Ready to freeze as a design baseline - with the meaning of freeze stated precisely. The five findings in this amendment were the known internal contradictions of the v0.1 to v0.3 text: each paired two statements that could not both be executed. With v0.4 applied, each pair has one reading, the rejected reading is named, and the surviving mechanism is specified at the level an implementer needs - predicates, keys, lock order, derivation rules and trigger context. No finding required a new table, a formula change or a reopened decision, which is evidence the architecture has stabilised: the corrections are now about the behaviour of mechanisms, not about which mechanisms exist. Freezing the schema design at v0.1 as amended by v0.2, v0.3 and v0.4 - 31 tables, the CL-01 status model, the CL-13 to CL-16 histories, and the CL-18 to CL-22 mechanisms - is therefore recommended, in this sense: *any further change requires a new targeted amendment with a demonstrated contradiction, exactly as v0.2, v0.3 and this amendment were produced.*

What freeze does not mean. A frozen design is not a verified design, and it is not an authorisation to build. Every mechanism listed under B-3 remains unexecuted; if the verification environment contradicts one of them - if, for example, trigger delta identification or the CL-22 visibility pattern does not behave as specified against a real PostgreSQL - the freeze is reopened for that mechanism by a further amendment, and the contradiction is recorded rather than worked around in code. Nor does freeze move any gate: market validation, merchant-confirmed evidence and explicit production-readiness approval are all still outstanding, and M1 production implementation remains gated behind them.

> **Final statement.** With this amendment applied, the specification no longer contains a test that its own trigger would reject (CL-18), a purge rule that could delete a live claim (CL-19), an anonymisation check that a concurrent insert can walk past (CL-20), a variance history that can cite another batch's money or present a stale calculation as current (CL-21), or a validation trigger whose ability to see its own evidence is assumed rather than specified and fail-closed (CL-22). Each resolution is at design level only, and this amendment says so in each finding rather than letting a described mechanism read as a proven one. The design is ready for schema freeze as a review baseline. It is not an authorisation to build.

End of amendment v0.4. This amendment supersedes only the sections it explicitly names. All other content of the Database Design Specification v0.1 and of Technical Correction Amendments v0.2 and v0.3 remains unchanged.